Password Manager

Petr Balok
Brno University of Technology
Faculty of Information Technology
Brno, Czechia
xbalok00@stud.fit.vutbr.cz

***Abstract*—This project uses best password management practices to implement a Password Manager with a graphical user interface (GUI) in Python.**

***Index Terms*—password manager, hashing, encryption, GUI**

I. Introduction

For this project, I decided to use the Python programming language to implement both the GUI and the password manager back-end. I also used the MySQL database to store the data since I have extensive experience with this system. I used the AES symmetric encryption algorithm with a 256-bit key and Argon2id for key derivation. For GUI, I chose Tkinter since it is bundled with Python, and there is no need to install many extra packages.

II. Hashing

There are many choices regarding (cryptographic) hashing or key derivation functions. There are functions like Message Digest 5 (MD-5), Secure Hash Algorithm (SHA), and key derivation functions such as bcrypt, Argon2, and PBKDF-2 [1]. Since there is a known collision in the MD-5 function [2], it is unsuitable for password hashing. The SHA-1 function is a cryptographic hash function that is also vulnerable because of a chosen prefix attack [3]. Then there is SHA-2, an improved version of the SHA-1 algorithm, but the NIST recommends using the SHA-3 function.

However, there are key derivation functions ideal for password hashing. These include functions like bcrypt and Argon2id. For my implementation, I used the Argon2id [6] algorithm, which won the 2015 password hashing competition [4]. I use this function to hash the master password and derive keys for the AES encryption to store passwords in the database. With its configurable parameters, the Argon2id function is quite robust for password hashing. The function has a large time complexity, making it harder for an attacker to quickly find the hashes of the most commonly used passwords. I also salt the master password and all encrypted passwords, so even if the user uses the same password multiple times (which the user should not do), Argon2id produces a different hash for each password, so the rainbow table attack [5] should not be possible.

In Python, I used the package argon2-cffi[1]. I generate my salt using Cryptodome Random[2] get_random_bytes function, which uses os.urandom, which is a cryptographically secure method of obtaining random bytes. Since I generate my salt, I used Argon2id low-level API to obtain hashes.

III. Encryption

Since this is a password manager, I must be able to retrieve the saved passwords from ciphertext back to plaintext for authentication in the services the user saved them for. I used a symmetric encryption function since it requires only one key, which is ideal for my use case.

The NIST approved two symmetric encryption functions, 3-DES and AES [7]. For my project, I used the AES encryption function in GCM mode. This mode produces ciphertext and message authentication code (MAC) to check integrity.

For AES encryption, I used the Cryptodome[3] package, which provides many cryptographic functions. Before starting the encryption, I derive the encryption key from the master password using Argon2id and a randomly generated salt. The key is 32 bytes (256 bits) long, which is recommended with AES encryption. After encrypting a password, I store the ciphertext as a base64 encoded string in the database along with salt, nonce (initial vector), and tag (password MAC).

IV. Database and authentication

I chose the MySQL database to store my data. In this database, there are two tables. One for the master password and one for encrypted passwords.

For the master password, I store only the salt, nonce (initial vector), tag (password MAC), and a test string. The test string is my chosen string that is encrypted using the master password hash. To correctly authenticate after the password manager setup, the user enters the master password. Then, a key is derived from the master password's hash, and the program tries to decrypt the testing string. If the decryption fails, the master password is incorrect. If it succeeds, the user is authenticated. The master password hash is stored only in memory during runtime.

The table for encrypted passwords contains columns for domain, username, password, salt, nonce, tag, and note.

- **domain** – Domain name of service the password is for
- **username** – Username for the service
- **password** – Base64 encoded string of a ciphertext of the password for the service

[1]https://argon2-cffi.readthedocs.io/en/stable/

[2]https://www.pycryptodome.org/src/random/random

[3]https://www.pycryptodome.org/src/cipher/aes

- **salt** – Base64 encoded random string added to the password
- **nonce** – Base64 encoded random initial vector for the AES function
- **tag** – Base64 encoded MAC of the password
- **note** – User's note for this entry

The base64 encoding is for easier conversion back to bytes data type in Python.

Therefore, in case of a database leak, the attacker cannot decrypt passwords stored in the database without knowing the master password.

To ensure a strong master password is being used, the user's master password should contain at least eight characters, and there should be at least one lower-case and one upper-case character, one digit, and one special character.

V. Graphical user interface

I used the Tkinter[4] package in Python to create the GUI. Since this package is bundled with Python, running this password manager with a GUI should be easy. In the GUI, I created multiple screens for the user to work with.

A. Login screen

Upon login, the user is asked to create a master password that complies with the password policy. After setting the password, the backend stores its salt in the database. Along with the salt, a test string is encrypted using the master password to authenticate the user in the future.

When the master password is set, the login screen is used for authentication. After entering the password, the backend tries to decrypt a test string from the database. If this encryption succeeds, the user is shown the main screen.

B. Main screen

On the main screen, entries for saved passwords are shown. In the columns, only domains, usernames, and notes for each entry are shown. To quickly copy the password for a given domain, the user can right-click on the row with that domain, and the password will be copied to the clipboard.

If the user left-clicks on a row, the entry edit screen appears. There are also buttons for other actions, such as adding a password, which takes the user to the entry addition form. There is also a logout button and a generator button, which takes the user to the password generator screen.

C. Password add form

When the user wants to add a password to the database, there are entry fields for domain, username, password, and note. If the user needs to check that the password is entered correctly, there is a show button to show the entered password. There is also a generate button, which generates a strong random password; the details of password generation are in the Generator screen subsection V-D. After filling out the form and pressing save, the password is encrypted using the methods described in the encryption section II. The cancel button shows the main screen without saving anything.

[4]https://docs.python.org/3/library/tkinter.html

D. Password edit form

If the user needs to see a saved password, copy it, or decides to change it, it can be done using the password edit form. After clicking a row on the main screen, the password edit form is shown with the same fields as in the password add form V-B with values retrieved from the database. The password fields contain the plaintext version of the saved password. There are buttons to copy and show the password, along with save and cancel buttons. The user can also generate a new password for a given domain.

E. Generator screen

On the password generator screen, the user can specify the length of a strong, randomly generated password. The length cannot be less than eight characters. For generating a strong random password, I used the secrets package[5]. This package is cryptographically secure for generating random values, including passwords. The generated password contains at least one upper and lower case character, a number, and a special character.

VI. Conclusion

The end result is a basic single-user password manager implemented using best practices for storing passwords. The password manager uses the Argon2id key derivation function to derive keys for encryption from the master password and the AES encryption algorithm to store all the passwords securely. The master password hash is not stored anywhere in the database, so to authenticate the user, the password manager tries to decrypt a test string that is encrypted in the database. If the decryption succeeds, the user is authenticated. After that, the user can perform all the required actions in the GUI to manage passwords for different domains and services.

[5]https://docs.python.org/3/library/secrets.html

References

[1] Authgear, "Password Hashing: How to Pick the Right Hashing Function," *Authgear Blog*, Feb. 6, 2023. [Online]. Available: https://www.authgear.com/post/password-hashing-how-to-pick-the-right-hashing-function

[2] P. Selinger, "Two different messages with the same MD5 hash," [Online]. Available: https://www.mathstat.dal.ca/~selinger/md5collision/ [Accessed: Apr. 19, 2025].

[3] G. Leurent and T. Peyrin, "SHA-1 is a Shambles: First Chosen-Prefix Collision on SHA-1 and Application to the PGP Web of Trust," in *Proceedings of the 29th USENIX Security Symposium (USENIX Security 20)*, USENIX Association, pp. 1839–1856, Aug. 2020. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/leurent

[4] OWASP Foundation, "Password Storage Cheat Sheet," *OWASP Cheat Sheet Series*. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html [Accessed: Apr. 19, 2025].

[5] B. Alhadawi and A. A. Yavuz, "A comprehensive evaluation of lightweight block ciphers for Internet of Things," EPFL, Lausanne, Switzerland, Tech. Rep., 2019. [Online]. Available: https://infoscience.epfl.ch/server/api/core/bitstreams/37cd827e-c3a6-4ba3-b68f-8ab7669b4449/content [Accessed: Apr. 19, 2025].

[6] A. Biryukov, D. Dinu, and D. Khovratovich, “Argon2: The memory-hard function for password hashing and other applications,” Tech. Rep., 2016. [Online]. Available: https://www.password-hashing.net/argon2-specs.pdf [Accessed: Apr. 19, 2025].

[7] National Institute of Standards and Technology (NIST), “Block Cipher Techniques,” *Computer Security Resource Center (CSRC)*. [Online]. Available: https://csrc.nist.gov/projects/block-cipher-techniques [Accessed: Apr. 19, 2025].